PROJECT REPORT

ON

WAREHOUSE ROBOT PATH PLANNING AND OBSTACLE AVOIDANCE SYSTEM

Submitted by

Abinaya M (26MCI10206)

in partial fulfillment for the award of the degree of

Master of Computer Applications in Artificial Intelligence and Machine Learning (MCA-AIML)

IN

Computer Applications



Chandigarh University

October - 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me in completing the project titled **"Warehouse Robot Path Planning and Obstacle Avoidance System."**

First, I would like to thank our respected Head of the Department and faculty members for providing me with the opportunity and necessary guidance to complete this project.

I am especially thankful to my project guide for their valuable suggestions, continuous support, encouragement, and guidance throughout the development of this project.

I would also like to thank Chandigarh University for providing the necessary facilities and learning environment to carry out this project successfully.

This project helped me gain practical knowledge of Design and Analysis of Algorithms, particularly Breadth-First Search (BFS), A* Search, path planning, and obstacle avoidance techniques using Python.

Finally, I would like to thank my parents, friends, and everyone who directly or indirectly supported me in completing this project successfully.

ABINAYA M

26MCI10206

INDEX

S.No.	Contents	Page No.
1	Introduction	3
2	Problem Statement	3
3	Objectives	3
4	Tools and Technologies Used	4
5	Methodology	4
6	Implementation	5
7	Implementation Flow	5
8	Output	6
9	Conclusion	8
10	Future Scope	9
11	References	10

1. Project Title

Project Title: Warehouse Robot Path Planning and Obstacle Avoidance System

Introduction

A warehouse contains storage shelves, packages, loading areas and narrow paths. Moving items manually or guiding robots without a planned route can take more time and may cause collisions. This project presents a grid-based path-planning system that helps a warehouse robot move from a starting point to a destination while avoiding blocked cells.

The warehouse is represented as a two-dimensional grid. Each cell represents a small area of the warehouse. Free cells can be travelled through, while blocked cells represent shelves, boxes, walls or other obstacles. The user selects the start and destination positions and runs a path-finding algorithm. The system calculates a safe route and displays it visually.

The project demonstrates important Design and Analysis of Algorithms concepts, including graph representation, state-space search, path cost, heuristic search, and comparison of algorithm performance.

2. Problem Statement and Objectives

Problem Statement

In a warehouse, a robot must travel from one location to another without passing through shelves, boxes or other blocked areas. Finding a suitable route manually is inefficient, especially when the warehouse layout changes. The problem is to design a system that finds a valid and preferably shortest route between a given start cell and destination cell while avoiding obstacles.

Objectives

- Represent the warehouse layout using a two-dimensional grid.
- Allow the user to identify the robot's starting point and destination.
- Represent shelves, boxes and other blocked areas as obstacles.
- Apply path-finding algorithms such as Breadth-First Search (BFS) and A* Search.
- Find a route that does not pass through obstacle cells.
- Display the calculated route clearly on the warehouse grid.
- Compare the route length and search effort of the selected algorithms.
- Recalculate a route when the warehouse layout or obstacle positions change.

3. Tools and Technologies Used

Tool / Technology	Purpose
Python	Programming language used to implement the path-planning logic.
Visual Studio Code	Code editor used to write, run and debug the project.
BFS Algorithm	Searches the grid level by level and finds a shortest path when every move has equal cost.
A* Search Algorithm	Uses actual path cost and a heuristic estimate to guide the search towards the destination.
NumPy (if used)	Can support grid and numerical data handling.
Matplotlib (if used)	Can be used to draw the warehouse grid, obstacles and paths.

4. Methodology

The system follows these steps:

1. Create a two-dimensional grid to represent the warehouse.
2. Mark the starting cell, destination cell and obstacle cells.
3. Treat each free cell as a graph node. Connect it to its valid neighbouring cells.
4. Generate possible movements in four directions: up, down, left and right.
5. Reject any movement that goes outside the grid or enters an obstacle cell.
6. Run the selected path-finding algorithm to search for a route.
7. Reconstruct the route by tracing the parent of each visited cell from the destination back to the start.
8. Display the route and its length. If no route exists, display a suitable message.

Algorithms

Breadth-First Search (BFS): BFS explores all neighbouring cells at the current distance before moving to the next distance level. For a grid in which every move has the same cost, BFS finds a shortest path. Its time complexity is O(V + E), where V is the number of cells (vertices) and E is the number of valid connections.

A* Search: A* selects the next cell using f(n) = g(n) + h(n). Here, g(n) is the cost from the start cell to the current cell, and h(n) is an estimated cost from the current cell to the destination. For a four-direction grid, Manhattan distance can be used as the heuristic: h(n) = |x1 - x2| + |y1 - y2|. With a suitable admissible heuristic, A* can find an optimal path while often exploring fewer cells.

Basic Pseudocode

```
START
  Read warehouse grid, start cell and destination cell
  Mark obstacle cells as blocked
  Select BFS or A* algorithm
  Add the start cell to the search structure
  WHILE the search structure is not empty:
     Select the next cell according to the algorithm
     IF the selected cell is the destination:
        Reconstruct and display the path
        STOP
     FOR each valid neighbour:
        IF neighbour is free and not visited with a better cost:
           Update its cost and parent
           Add it to the search structure
  Display 'No path found' if destination was not reached
END
```

5. Implementation

The project is implemented in Python. The warehouse is stored as a grid in which free cells are available for movement and blocked cells are treated as obstacles. The program receives the start and destination coordinates and applies the selected search algorithm.

Main Modules

- Grid Creation: creates the warehouse layout and defines its rows and columns.
- Start and Destination Selection: identifies the robot's initial position and target position.
- Obstacle Handling: marks blocked cells and prevents the robot from moving through them.
- Path Finding: executes BFS or A* to search for a valid route.
- Path Reconstruction: uses stored parent information to rebuild the route.
- Path Visualization: displays the start, destination, obstacles and calculated route using different visual styles or colours.

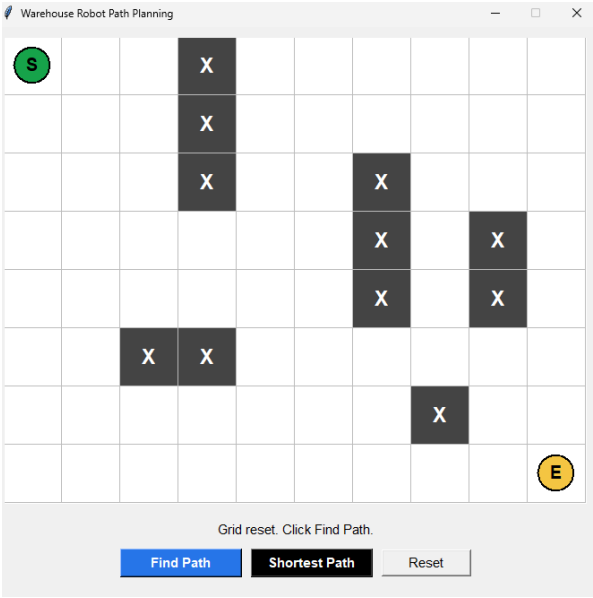
Implementation Flow

The user opens the application, checks the warehouse grid, sets or confirms the start and destination, and places obstacles where required. After clicking the Find Path control, the program performs the search and displays the resulting route. When the layout changes, the search can be performed again to obtain an updated route.

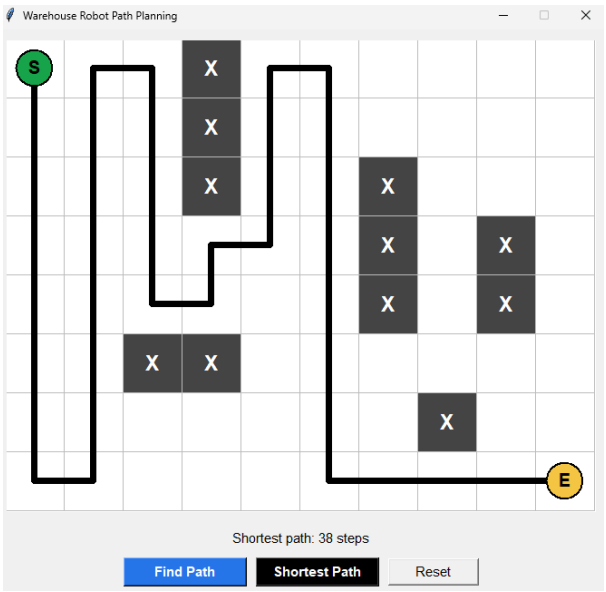
If the interface is configured to compare multiple candidate routes, each candidate may be displayed in a different colour and the shortest route may be highlighted in black. The exact display behaviour should match the final running version of the program.

6. Output

Main Warehouse Grid

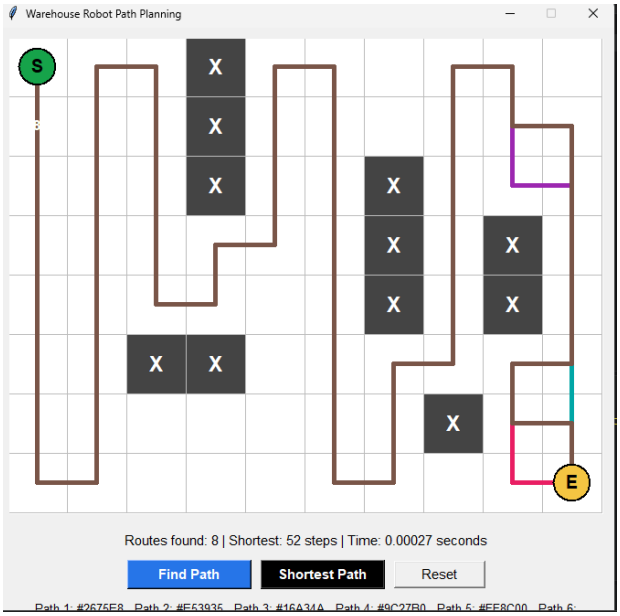


Find Path Result



t

3: Shortest Path



.

6.Result

The system demonstrates how a warehouse robot can navigate through a grid while avoiding blocked cells. BFS can be used to obtain a shortest path when all movements have equal cost, while A* uses a heuristic to guide the search. The final result depends on the warehouse layout, obstacle positions and selected algorithm.

7. Conclusion

The Warehouse Robot Path Planning and Obstacle Avoidance System demonstrates the use of graph-based search algorithms in a practical warehouse navigation problem. By representing the warehouse as a grid, the system can identify free and blocked areas and calculate a route from the robot's starting point to its destination.

The project helps understand BFS and A* Search, graph traversal, path reconstruction and obstacle handling. It can be extended in the future by supporting weighted paths, dynamic obstacles, multiple robots, real-time sensor input and automatic route updates. Thus, the project provides a foundation for intelligent and efficient warehouse robot navigation.

8.Future Scope

The Warehouse Robot Path Planning and Obstacle Avoidance System can be improved in several ways in the future. The following features can make the system more intelligent, efficient, and suitable for real-world warehouse applications.

1. Real-Time Obstacle Detection: Integrate sensors or cameras to detect obstacles automatically while the robot is moving.

2. Dynamic Path Planning: Allow the robot to change its route whenever a new obstacle appears in its path.

3. Multiple Robot Coordination: Extend the system to support multiple robots working together in the same warehouse without collisions.

4. AI-Based Navigation: Use Artificial Intelligence and Machine Learning techniques to improve navigation and decision-making.

5. Shortest Path Optimization: Implement advanced algorithms such as Dijkstra's Algorithm and A* Search to find efficient routes.

6. Live Warehouse Monitoring: Develop a real-time dashboard to monitor robot position, movement, obstacles, and task progress.

7. Automatic Charging System: Enable robots to identify low battery levels and automatically move towards charging stations.

9. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). Introduction to Algorithms, 4th Edition, MIT Press.
- Python Software Foundation. Python 3 Documentation. Available at: https://docs.python.org/3/
- Hart, P. E., Nilsson, N. J., and Raphael, B. (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." IEEE Transactions on Systems Science and Cybernetics.
- Russell, S. and Norvig, P. Artificial Intelligence: A Modern Approach, 4th Edition, Pearson.
- Python Software Foundation. Heap Queue Algorithm (heapq) Documentation. Available at: https://docs.python.org/3/library/heapq.html
- Python Software Foundation. Collections – Deque Documentation. Available at: https://docs.python.org/3/library/collections.html
- NumPy Developers. NumPy Documentation. Available at: https://numpy.org/doc/
- Matplotlib Development Team. Matplotlib Documentation. Available at: https://matplotlib.org/stable/